

CSCI 6360 Parallel Computing

Assignment 2: 1D Stencil on Multiple GPUs

Quinten Pienaar

662041924

pienaq@rpi.edu

Due: March 10, 2026

1 Overview

This assignment examined the effect of utilizing multiple GPUs to compute a 1D stencil algorithm. Performance was measured via execution time across a range of grid sizes and device configurations. Unified memory and prefetching was employed to facilitate efficient memory access across devices. Results demonstrate substantial performance improvement in the two-device configuration relative to a single device, with diminishing returns observed in the four-device case. Grid size had negligible impact on execution time within any given device configuration.

2 Testing

All experiments were conducted on a single AiMOS compute node using the IBM POWER9 architecture. The program was invoked with the following command:

```
./hw2 NumElements HaloSize NumThreadsPerBlock GridSize MemPrefetch NumDevices
```

A total of 15 tests were conducted with the goal of determining how grid size and device count affects execution time. Accordingly, tests varied grid size and number of devices, but kept all other parameters constant. The test cases and their parameters are given in Table 1 and Table 2.

Table 1: Fixed Experimental Parameters

Parameter	Value
NumElements (exponent)	30 (2^{30} elements)
HaloSize	64
ThreadsPerBlock	1024
MemPrefetch	1 (enabled)

Table 2: Varied Experimental Parameters

Parameter	Values Tested
GridSize	1024, 2048, 4096, 8192, 16384
NumDevices	1, 2, 4

Execution time was measured using the `getticks()` function, which reads the IBM POWER9 CPU cycle counter at a resolution of 512,000,000 ticks per second. The timed region included memory prefetching and kernel execution, but excluded array allocation and initialization. All 15 cases produced a `SUCCESS` result, confirming correctness across every configuration.

3 Implementation

In contrast to prior work, which focused on developing a parallelizable stencil algorithm for a single device, this experiment focused on coordinating computation across multiple GPU devices. Several key areas of implementation are described below.

3.1 Memory Management

The input and output arrays were allocated using `cudaMallocManaged`, making both arrays accessible across all GPUs through a single unified pointer. Memory prefetching via `cudaMemPrefetchAsync` was enabled in all experiments to avoid costly on-demand page fault migrations.

3.2 Work Partitioning

The input array was divided evenly across devices, with each device assigned a contiguous chunk of the input of size `chunk_size = N / NumDevices`. To ensure efficient stencil computation at chunk boundaries, each

device was prefetched an additional HALO elements beyond its assigned region. Elements outside the bounds of the global array were treated as zero, preserving correct stencil behavior at the array edges.

3.3 Kernel Launch

After chunk sizes were determined, the execution timer was started. A loop then selected the target device via `cudaSetDevice`, prefetched the appropriate slice of both the input and output arrays, and launched the stencil kernel over that device's chunk. Critically, all kernel launches were issued before any synchronization, allowing devices to execute in parallel. Only after all kernels had completed their stencil calculation was the execution timer stopped.

Algorithm 1 Multi-GPU Kernel Launch

```

1: chunk_size ← size of input array / number of devices
2: start ← getticks()
3: for device in Devices do
4:   cudaSetDevice(device)
5:   begin ← where the device should begin to work on input array including edges
6:   stencil_size ← size of input array to work on including edges
7:   output_index ← where the device should write to the output array
8:   cudaMemPrefetchAsync(&input[begin], size, device)
9:   cudaMemPrefetchAsync(&output[output_index], chunk_size, device)
10:  stencil<<<(numBlocks, TPB)>>>()
11: end for
12: cudaDeviceSynchronize()
13: end ← getticks()

```

3.4 Validation

Validation was performed sequentially by traversing the output array and comparing each element against its analytically expected value. Given that all input elements are initialized to 1, the expected output at index i for an array of size N with halo radius H is:

$$y[i] = \begin{cases} 1 + H + i & \text{if } i < H \\ 2H + 1 & \text{if } H \leq i < N - H \\ N - i + H & \text{if } i \geq N - H \end{cases} \quad (1)$$

Interior elements accumulate a value of $2H + 1$ if the input array is uniformly equal to 1, yielding $2H + 1 = 129$ for $H = 64$. Boundary elements accumulate a smaller value as the stencil window extends beyond the array, causing the expected value to taper linearly toward the edges.

4 Experimental Results

Table 3 presents the execution time in seconds for each combination of grid size and device count. Figure 1 plots these results for visual comparison.

Table 3: 1D Stencil Execution Time (seconds)

Grid Size	1 GPU	2 GPUs	4 GPUs
1024	1.030510	0.304224	0.528427
2048	1.035271	0.289754	0.514440
4096	1.017864	0.282682	0.533898
8192	0.997656	0.284514	0.522045
16384	0.998818	0.279212	0.535847
Best	0.997656	0.279212	0.514440

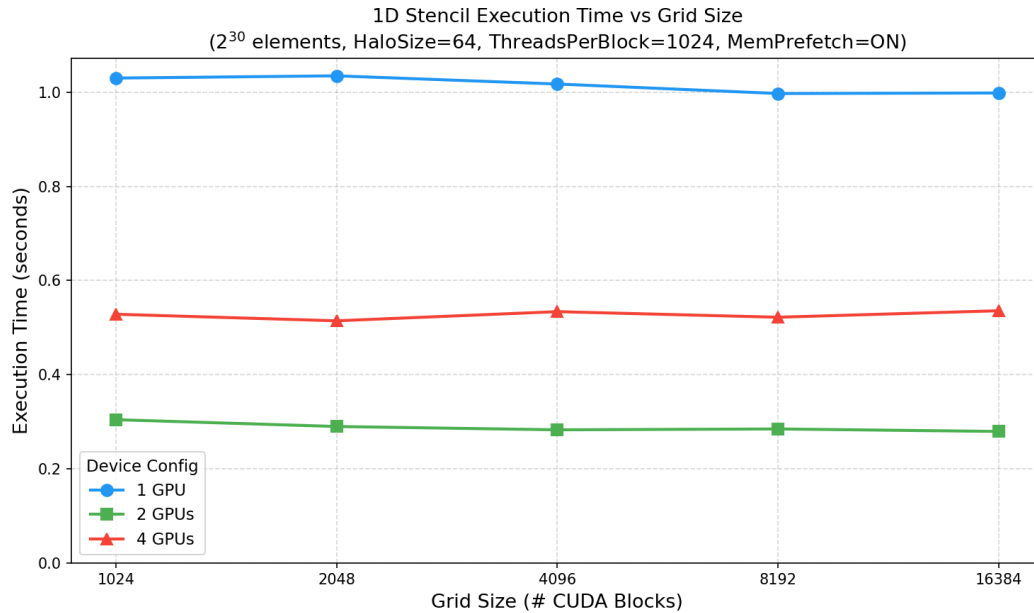


Figure 1: 1D Stencil execution time vs. grid size for 1, 2, and 4 GPUs.

5 Performance Analysis

The 2-GPU configuration achieved the best performance across all grid sizes, with a best execution time of 0.279 seconds at a grid size of 16384 — a speedup of approximately $3.58\times$ over the best single-GPU time of 0.998 seconds. This super-linear improvement is attributable to the combined effect of parallel execution and proper prefetching, which allows both devices to migrate and process their data efficiently.

Grid size had negligible impact on execution time across all device configurations. This indicates that the bottleneck lies in memory bandwidth and data migration rather than kernel occupancy or compute throughput.

The 4-GPU configuration, while faster than the single-GPU baseline, underperformed relative to the 2-GPU case. This suggests that at a problem size of 2^{30} elements, the overhead associated with coordinating four devices outweighs the benefit of further parallelism.

To determine whether the 4-GPU configuration becomes advantageous at larger problem sizes, additional experiments were conducted with 2^{31} and 2^{32} elements. The results, included in Appendix A, show that execution times remained nearly identical across all three problem sizes for every device configuration. This indicates that scaling the problem up to $4\times$ the original size is insufficient to amortize the coordination

overhead of 4-GPUs, and that the 2-GPU configuration remains optimal.

Compared to a comparable single-GPU experiment from Assignment 1, grid size 1024, 1024 threads per block, prefetch enabled, the current single-GPU result is faster. This improvement is attributed to a corrected prefetching order in the current implementation: in Assignment 1, `cudaMemPrefetchAsync` was called prior to array initialization, causing the CPU writes to invalidate the prefetched pages and forcing on-demand page fault migrations at kernel launch. In the current implementation, prefetching occurs after initialization, ensuring a clean transfer to the GPU before execution begins.

6 Conclusions

This assignment demonstrated that distributing a 1D stencil computation across multiple GPUs yields substantial performance improvements over a single-GPU baseline. The 2-GPU configuration was the most effective, achieving a $3.58\times$ speedup over the single-GPU case with a best execution time of 0.279 seconds. The 4-GPU configuration, while faster than a single GPU, failed to improve upon the 2-GPU result, suggesting that the overhead of coordinating four devices outweighs the benefit of distributing the workload further at this problem scale.

Grid size and problem size both had a negligible impact on performance across all device configurations. This poses a difficult question: how could performance be improved from current results? Flat performance across grid size indicates that the workload is memory-bound, and that execution time should scale proportionally with problem size. However, execution time remained nearly flat across 2^{30} , 2^{31} , and 2^{32} elements. This then implies that compute is the performance bottleneck.

However, if compute was the bottleneck, the 4-GPU configuration would be expected to outperform 2 GPUs, which was not observed. The data therefore does not cleanly support either characterization, and the true bottleneck is ambiguous.

Future work should be done to explore the nature of the performance bottleneck. This could be done by conducting experiments with prefetching disabled, which would reveal how much of the transfer cost is currently being hidden by `cudaMemPrefetchAsync`. Additionally, varying the halo radius across a wider range would change the ratio of memory accesses to output elements — if execution time scales proportionally with halo size, the workload is memory-bound; if it remains flat, the bottleneck lies elsewhere.

A Extended Experiments

The following results extend the primary experiment to larger problem sizes of 2^{31} and 2^{32} elements. All 30 additional test cases produced a SUCCESS result.

A.1 2^{31} Elements

Table 4: 1D Stencil Execution Time (seconds) — 2^{31} Elements

Grid Size	1 GPU	2 GPUs	4 GPUs
1024	0.993683	0.299373	0.538628
2048	1.024552	0.291879	0.510500
4096	1.014666	0.291148	0.536482
8192	1.016376	0.283316	0.520634
16384	1.005617	0.278931	0.579348
Best	0.993683	0.278931	0.510500

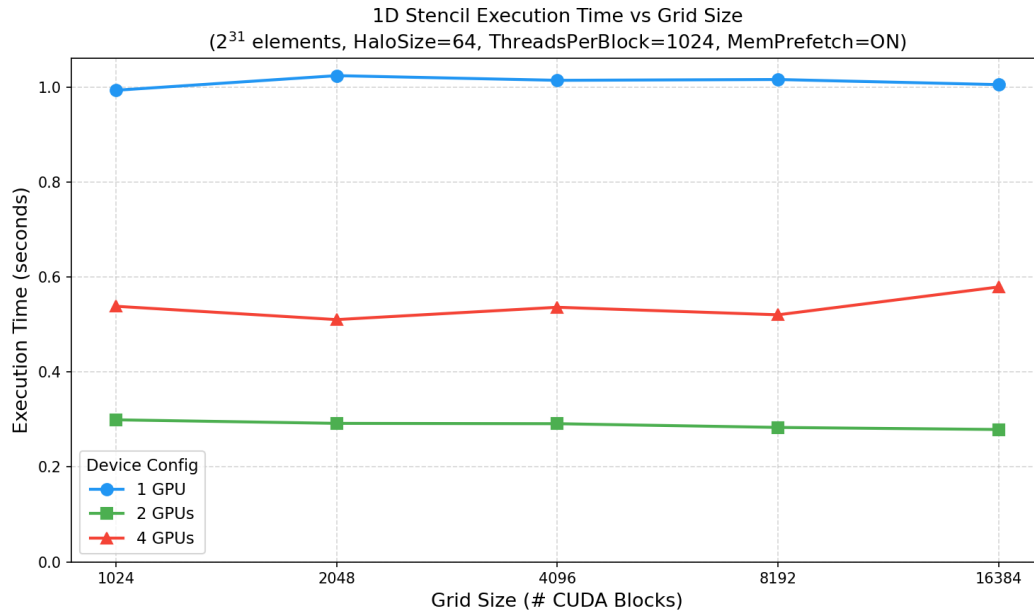


Figure 2: 1D Stencil execution time vs. grid size for 1, 2, and 4 GPUs (2^{31} elements).

A.2 2^{32} Elements

Table 5: 1D Stencil Execution Time (seconds) — 2^{32} Elements

Grid Size	1 GPU	2 GPUs	4 GPUs
1024	1.053806	0.301375	0.556170
2048	1.020619	0.291631	0.536753
4096	1.031574	0.285422	0.533919
8192	0.984412	0.282191	0.518396
16384	0.988295	0.281437	0.515589
Best	0.984412	0.281437	0.515589

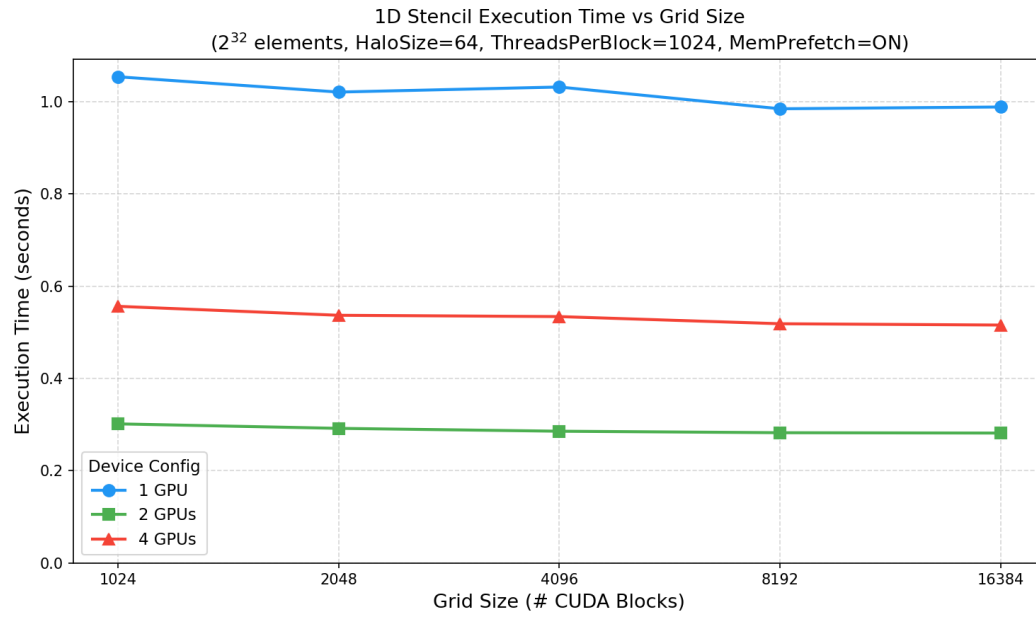


Figure 3: 1D Stencil execution time vs. grid size for 1, 2, and 4 GPUs (2^{32} elements).